Three-assistant comparison and role decision

Date: 8 October 2026

Status: Perplexity recommendation for user approval; not a joint decision approved by ChatGPT and Claude.

1. Material actually reviewed

ChatGPT: attached paste.txt, including the conversation and a detailed daily-plan document-generation script. This is not verification that its claimed Word document exists or that any repository code works.

Claude: responses pasted by the user. The full Claude daily-plan document was not supplied, so its daily tasks and complete gap coverage cannot be audited.

Perplexity: responses and the previously generated daily plan in this conversation.

No source repository, CV, independent coding assessment or production deployment was audited. Claims about 23 years of experience, 40+ products or senior readiness remain source claims, not independently established facts.

2. Comparison

ROLE

ChatGPT: Senior AI / Agentic Systems Engineer; explicitly uses prior data-engineering background as an advantage.

Claude: AI Engineer focused on LLM and agentic systems; Applied ML Engineer second.

Perplexity: Applied AI / AI systems / agentic application engineer; applied ML only after a separate gate.

Finding: substantial convergence on building production LLM/agent systems, not conventional data-engineering employment. Seniority is the main unresolved difference.

CURRICULUM

ChatGPT: strongest explicit breadth in the supplied material: RAG, agents, auth/OIDC, tenancy, security, observability, Azure, CI/CD and limited React/TypeScript. Risk: too much scope for the available time, and a senior title before demonstrated senior AI ownership.

Claude: clearest independent-coding rule: learner types the code; assistant reviews and quizzes. It reports 21.5 hours/week, later 22.75, but the underlying daily document is unavailable. Risk: a blanket prohibition on generated code may not suit every lesson; it is Claude's proposed method, not a rule imposed on all teachers.

Perplexity: explicit diagnostic, Python/SQL/API foundations, AI evaluation, a small classical ML project, remediation gates and a 130-hour calendar. Risk: generic foundation work and the ML mini-project could consume time without resolving the most important role-specific weaknesses.

LEARNING FREEDOM

All three initially attempted to synchronise or prescribe the teaching process. The user's correction overrides those proposals: common career destination, independent teaching methods, no fixed assistant roles and no compulsory identical syllabus. Preserve proposals and learning evidence separately.

REPOSITORIES AND CONTEXT PROJECT

ChatGPT: ai-engineering-portfolio and ai-engineering-workbench; public evidence derived from private working truth; incremental retrieval architecture.

Claude: supports the workbench/tutor records, but no final two-repository naming or complete architecture is present in the supplied excerpt.

Perplexity: context-ledger and context-ledger-workspace; raw/normalised/derived records, provenance and bounded CLI-first scope.

Finding: use career-aligned repository names, with ContextLedger as a provisional project name inside them. This keeps the public repo useful beyond one product.

CLAIMS THAT NEED CORRECTION

Claude's 'most in-demand title' and 2-3 month hiring timeline are not established by the supplied response. Do not use them as guarantees.

ChatGPT's senior target is an aspiration, not a demonstrated level. Past experience is relevant but does not establish senior AI competence automatically.

Perplexity's four-hour plan is a planning assumption, not proof that the user can sustain it or that 130 hours closes all gaps.

Three models agreeing is not independent verification. Actual tests, source evidence and the user's work remain necessary.

3. Role decision

Recommended primary target: AI Engineer - LLM and Agentic Systems.

Equivalent search titles: Applied AI Engineer; Agentic AI Engineer; LLM Engineer; AI Systems Engineer, when the duties actually match.

Use title variants to find the same job family, not to redirect the user into data engineering.

Secondary conditional target: Applied ML Engineer, only where the model-training/statistics requirements match demonstrated skills.

Excluded targets: Data Engineer, Analytics Engineer, warehouse-focused roles, data-platform roles relabelled as AI, and prompt-only work without substantive engineering ownership.

Level: evidence-led. Do not automatically label the user junior or senior. Assess Python independence, architecture judgement, production reliability, delivery ownership and communication against each vacancy. Senior applications remain possible when prior experience plus AI evidence satisfy that vacancy.

Application timing: prepare now; decision checkpoint around 15 November, conditional on the outcome the user is waiting for and the user's decision. No applications are authorised here.

4. Why this target

It is the clearest intersection of the three proposals and the user's AI/ML/agentic boundary. It aligns the context-workbench project with retrieval, APIs, tools, evaluation and reliable deployment.

A reviewed JPMorganChase Applied AI Engineer - Agentic Systems advertisement includes RAG, tools/APIs, guardrails, tracing, evaluations and production delivery. It also lists demanding qualifications and ML knowledge; it is evidence of the role family, not proof of the user's eligibility.

Reviewed ML listings include statistics, optimisation, model evaluation, framework proficiency and deployment. Therefore applied ML is not automatically interchangeable with LLM application engineering.

No salary estimate, title-demand ranking or hiring-duration promise is made.

5. Shared destination, independent teaching

Each assistant owns its proposed teaching path and can change it with an explanation. Keep plans under separate teacher directories. A role-target decision does not merge the three curricula.

A minimal shared evidence contract records: objective, learner attempt, assistance used, actual commands/results, understanding, blockers and next action. No fixed teaching order, equal-time quota or requirement that all teach the same topic each day.

The earlier schedules must not be added together as a commitment. Track actual time across all three paths during the first week, then negotiate scope with the user. Preserve repeated explanations when useful; remove accidental duplicate engineering work rather than automatically removing repeated learning.

6. First readiness measurements

1. Explain one existing AI system and identify personal decisions versus AI-generated output.

2. Complete a small Python/API change and tests independently using ordinary documentation.

3. Diagnose a bad retrieval answer from evidence.

4. Explain agent versus deterministic workflow and safe tool execution.

5. Run and interpret an evaluation set including failures.

6. Explain a clean deployment and one incident/recovery path.

Use results to revise gaps. Do not invent percentage readiness scores.

7. Reference project scope

Provisional project name: ContextLedger. Name availability and trademark status have not been checked.

First version: manual/file import; originals plus normalised records; lexical search; approved-decision tracking; source-linked context packs; Git session handovers; CLI and tests; a synthetic public demo.

Next, only if a learning task requires it: semantic retrieval, one read-only API/tool integration, authentication, telemetry or a thin dashboard.

Deferred: automatic capture from all applications, multi-client integration promises, desktop application, commercial multi-tenancy, model fine-tuning and SLM training.

Retrieval failures and assistant failures must remain distinguishable. Import completeness, index freshness, retrieved sources and context-pack omissions are reportable evidence, not claims that hallucinations are solved.

8. Publication and privacy

Public examples are synthetic or explicitly reviewed and redacted. Never publish the private workspace by changing its visibility. Publish selected files into the public repository with provenance and a review record; do not merge private history.

No secrets, credentials, customer data or employer-confidential content in either Git repository. Exclude embeddings, runtime databases and caches from Git; store reproducible indexing configuration instead. Raw conversation commits require a sensitivity review. Personal material may need protected storage rather than Git; keep safe references/manifests when necessary.

9. Approval boundary

Files are local proposed deliverables. No GitHub repository was created, no branch pushed, no existing normal Git protocol inspected, and no joint approval claimed.

Before remote setup: user approves names, visibility and skeleton; resolve the GitHub owner and inspect the existing protocol. External creation or writes require a separate exact-action confirmation.

Sources

ChatGPT material: user attachment paste.txt.

Claude material: user-pasted response in this conversation.

Perplexity material: this conversation and prior generated plan.

Job evidence consulted 8 October 2026:

https://builtinlondon.uk/job/applied-ai-engineer-agentic-systems-senior-associate/10308557

https://www.itjobswatch.co.uk/find/PyTorch-jobs-in-UK

Proposed two-repository Git structure

Date: 8 October 2026

Status: proposed; no repositories created.

Names and visibility

ai-engineering-portfolio: public when reviewed and ready; external showcase and reusable demonstrator code.

ai-engineering-workbench: private; working sessions, learning paths, decision history and private configuration references.

ContextLedger is the provisional project name, not a required third repository.

Public repository

text

ai-engineering-portfolio/

README.md

LICENSE # select explicitly; not assumed

SECURITY.md

CONTRIBUTING.md

AGENTS.md

.gitignore

.env.example # placeholders only

pyproject.toml

src/context_ledger/ # single source of reusable product code

ingest/

records/

search/

context/

cli/

tests/

examples/synthetic/

schemas/

evaluations/

synthetic_cases/

reports/

docs/

architecture/

decisions/

limitations.md

case-studies/

demo-guide.md

.github/

workflows/ci.yml

pull_request_template.md

README documents problem, scope, setup, actual demo status, limitations and personal ownership. No actual private conversation in examples or CI fixtures.

Do not fabricate a public 'production' claim. A local CLI is a local CLI until deployment is demonstrated.

Private repository

text

ai-engineering-workbench/

README.md

AGENTS.md

.gitignore

protocol/

existing-git-turn-protocol.md # import actual protocol; do not invent it

session-start.md

session-end.md

publication-checklist.md

role-target/

decision.md

vacancy-matrix.csv

gap-register.csv

readiness-rubric.md

teachers/

chatgpt/

proposal.md

plan.md

lessons/

exercises/

assessments/

claude/

proposal.md

plan.md

lessons/

exercises/

assessments/

perplexity/

proposal.md

plan.md

lessons/

exercises/

assessments/

learner/

attempts/

reflections/

time-log.csv

evidence-index.csv

conversations/

manifests/

raw/ # only sensitivity-reviewed captures

normalized/

sessions/YYYY/MM/DD/

context/

current-state.md

approved-decisions.md

open-questions.md

handovers/

decisions/

reviews/

evidence/

applications/ # private CV/application drafts

publications/ # manifests, approvals, public commit links

config/

corpus.example.yaml # no credentials or raw sensitive values

Runtime databases, indexes and caches are outside tracked paths. Public code reads a private corpus through an explicit local path or protected service; the public repository never embeds the private corpus.

Code ownership boundary

Reusable product implementation lives once, in the public repository after safety review; develop private-first if necessary until safe publication. Workspace holds private records and configuration, not a second diverging copy of product code.

Record the exact product commit/release used for private runs. Two clones can be adjacent locally; do not silently mix their histories. If code is not safe for public release, keep a documented private staging area and publish selected clean snapshots only after approval.

Proposed Git workflow extension

The existing user protocol governs. These are requirements to reconcile with it, not a claim that it was inspected.

1. Start: record repository, branch, HEAD, git status, task and files actually read.

2. Branch by task and assistant, for example docs/perplexity-role-review or feat/chatgpt-context-cli.

3. One writer for a given implementation task; other assistants can teach independent lessons and review.

4. Scoped changes; no destructive cleanup, force push, dependency churn or cross-repository copying without agreement.

5. Verify with actual commands. Distinguish not run, failed and passed.

6. Open review/PR with learner ownership and assistant assistance recorded.

7. Merge through the user's normal approval protocol.

8. End with status, commit, evidence, unresolved issues and handover.

Naming examples are conventions to approve, not mandatory replacements for existing branch rules.

Session record template

session_id:

assistant:

project:

objective:

repository:

branch:

starting_commit:

files_actually_read:

context_pack_id:

learner_attempt:

assistance_received:

changes:

commands_run:

observed_results:

commands_not_run:

understanding_demonstrated:

unresolved_questions:

decisions_proposed:

decisions_approved_by_user:

ending_commit:

next_action:

Context provenance rules

Original captures are retained where safe. Each normalised message has a stable ID, platform, role, timestamp status, capture method and original-source reference.

Summaries are derived records. They link to sources and do not overwrite originals.

Assistant statements remain claims until verified. Suggestions remain proposed until accepted. Corrections link to what they supersede.

A context pack identifies corpus version, relevant sources, decisions, conflicts, omissions and missing imports.

No assistant may say it has read another assistant's session unless it actually accessed that record.

Public publication checklist

- Explicit allowlist of files; no whole-workspace synchronisation.
- Inspect content and history for secrets and personal information.
- Review synthetic fixtures for accidental real data.
- Confirm rights to publish any third-party or employer material.
- Preserve accurate attribution and test-status labels.
- Record private source references only in private publication manifest.
- Public case study links to safe public commits, not private sessions.
- Obtain approval for remote changes.

Initial sequence

1. Approve the role-target recommendation and repository names separately.
2. Resolve GitHub account/organisation and inspect actual Git-turn protocol.
3. Agree visibility and initial files.
4. Request exact-action confirmation before remote creation/writes.
5. Import supplied conversation material with capture/completeness labels.
6. Start independent teacher proposals and a learner baseline.
7. Build the smallest searchable, source-linked context CLI.